

Laboratory Activity No. 1

Laboratory Report

Real-Time Multisensor Room Monitoring System

BCA152 Microcontrollers

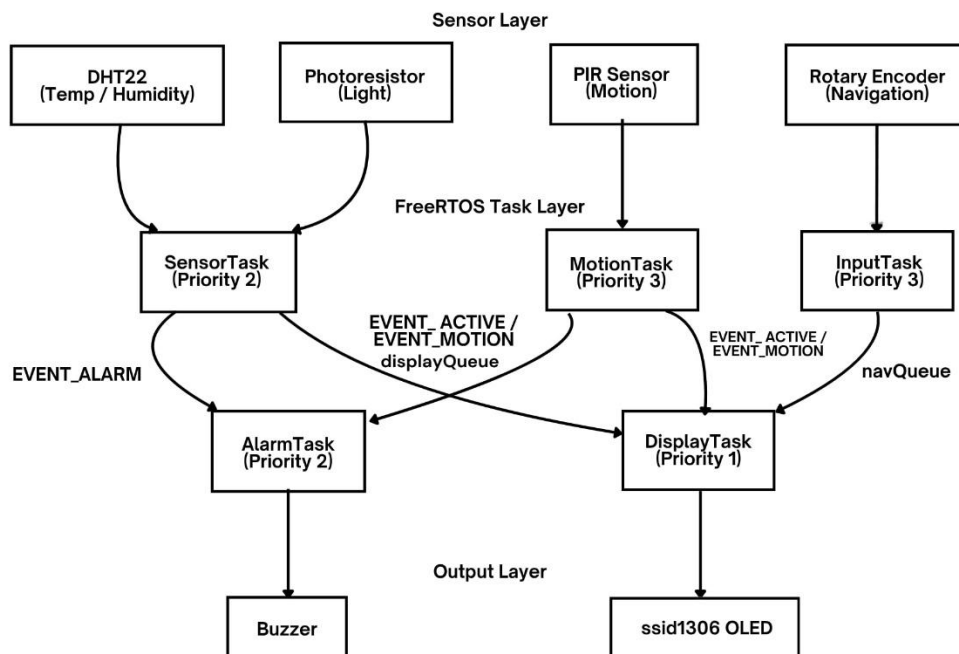
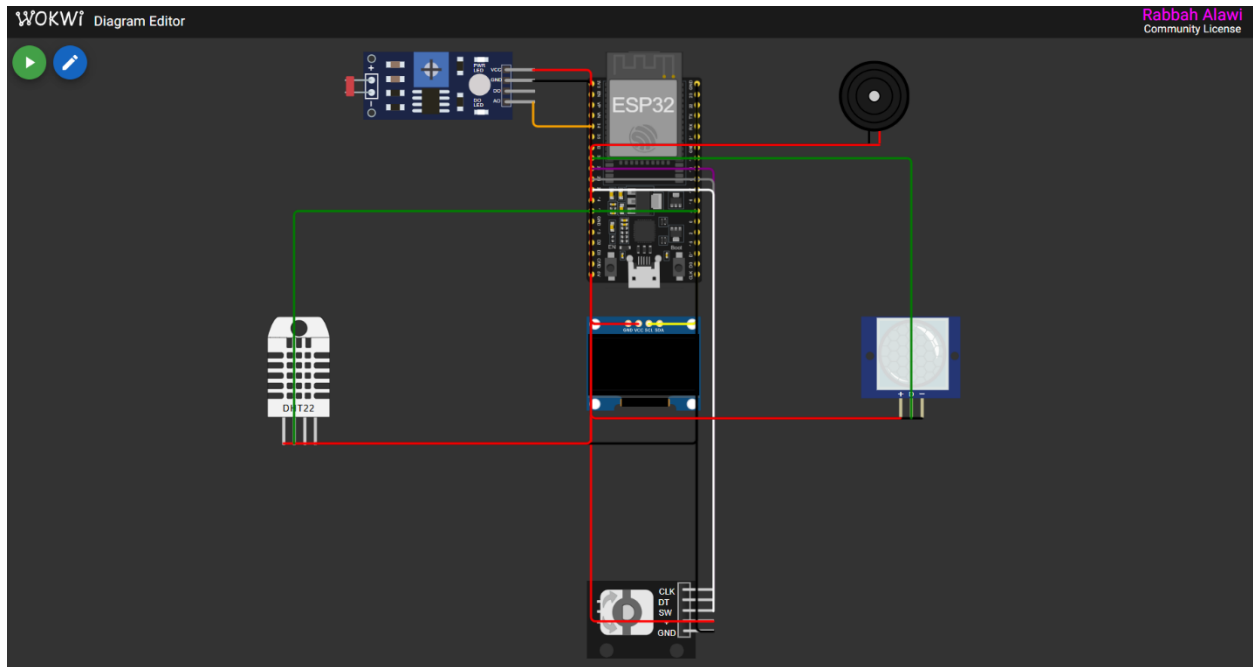
Mindanao State University – Iligan Institute of Technology
College of Computer Studies, Department of Computer Applications

Rabbah Alawi

1. Problem and Requirements

The problem is that this lab required to design a FreeRTOS based ESP32 system that monitors a room temperature, humidity, light and motion, and it needs to be display in OLED with buzzer. Since there's no hardware devices, it is required to use Wokwi that will act as a real hardware.

2. System Architecture and Design



3. FreeRTOS Architecture

Priority Is Scheduling Urgency

- MotionTask and InputTask (Priority 3) – these handle the physical events, because a missed encoder pulse or delayed motion trigger can't be recovered later, so they need to run immediately.
- SensorTask and AlarmTask (Priority 2) – these tasks read slowly changing data, so a short delay doesn't hurt anything.
- DisplayTask (Priority 1) – only updates a screen humans read at their own pace, so it's okay to have delay.

If DisplayTask is the high priority instead, its slow I2C writes would block the time critical tasks and cause missed encoder clicks or delayed motion response.

3.1 Task Overview

Task	Priority	Period/Trigger	Responsibility
MotionTask	3	100 ms polling	Monitor PIR, manage ACTIVE/INACTIVE transitions
InputTask	3	10 ms polling	Read rotary encoder, send navigation direction
SensorTask	2	2 s (vTaskDelayUntil)	Read DHT22 + LDR, evaluate alarm condition
AlarmTask	2	Event-driven (EVENT_ALARM)	Drive buzzer based on alarm state
DisplayTask	1	~100 ms loop	Own OLED, render current display mode

3.2 Inter-Task Communication

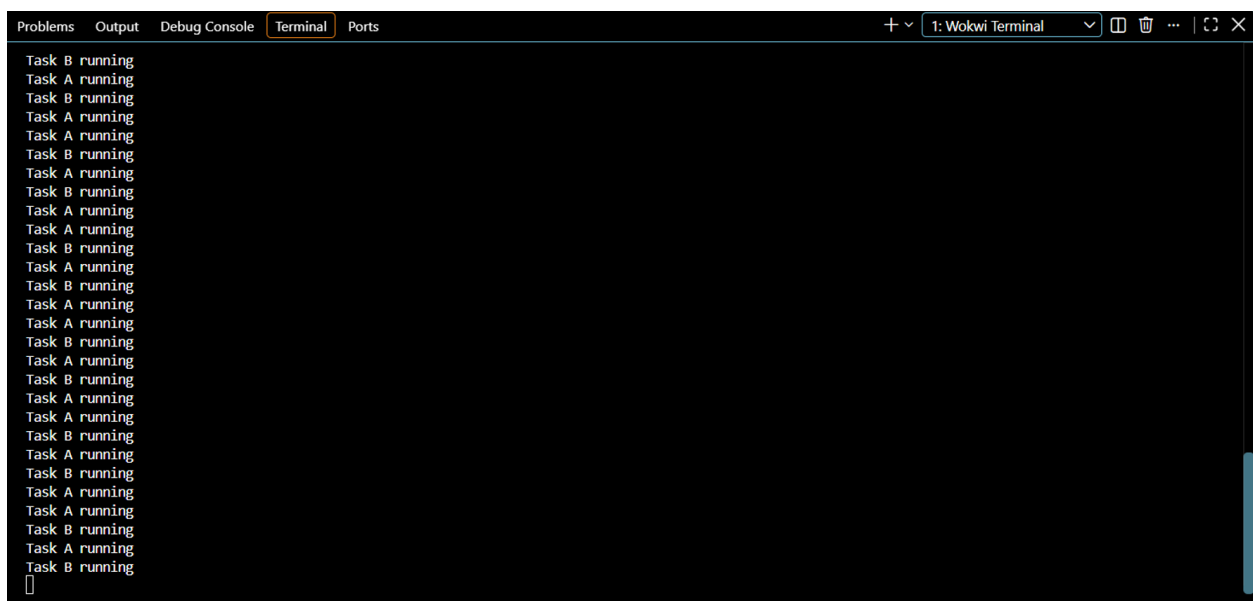
- displayQueue – carries SensorData (temperature, humidity, light, motion) from SensorTask to DisplayTask and AlarmTask.
- navQueue – carries NavDirection from InputTask to DisplayTask.
- g_systemEvents (event group) – bits EVENT_ACTIVE, EVENT_MOTION, EVENT_ALARM, set/cleared by MotionTask/SensorTask, read by DisplayTask/AlarmTask.

Why vTaskDelayUntil() for Periodic Sensor Sampling

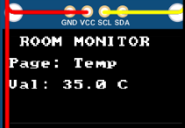
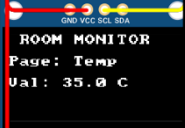
vTaskDelay() counts the delay starting from when it's called, so any time the sensor read takes gets added on top over many cycles, this makes the sampling interval slowly drift longer. vTaskDelayUntil() tracks a fixed absolute wake-up time instead, so it keeps SensorTask sampling at exactly 2 seconds every time, with no drift.

Required Mutex Explanation

The shared resource is the serial/UART output (printf), used by both SensorTask and AlarmTask. Without the mutex, if both tasks print at nearly the same moment, FreeRTOS could switch between them mid-print, garbling the two messages together into unreadable text. serialMutex prevents this by forcing one task to finish printing completely before another can start.



Required Functional Tests

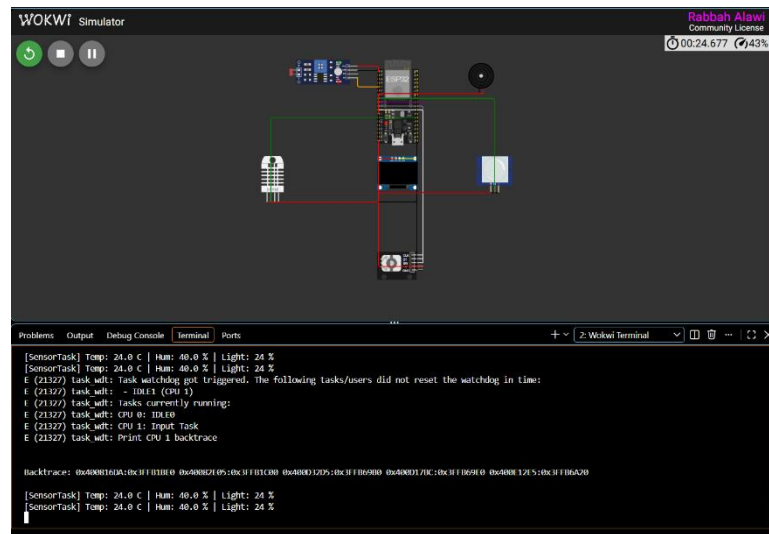
Test ID	Input/Stimulus	Expected	Actual	Result
FT-01	Change temperature	Displayed temperature updates.		PASS
FT-02	Change humidity	Displayed humidity updates.		PASS
FT-03	Change light input	Light value changes.		PASS
FT-04	Rotate encoder clockwise	Next page is selected.		PASS
FT-05	Rotate encoder counterclockwise	Previous page is selected.		PASS
FT-06	Set temperature above 30°C	Alarm activates	[SensorTask] Temp: 24.0 C Hum: 40.0 % Light: 24 % [SensorTask] Temp: 30.7 C Hum: 40.0 % Light: 24 % [AlarmTask] Alarm state changed -> ACTIVE [SensorTask] Temp: 30.7 C Hum: 40.0 % Light: 24 % [SensorTask] Temp: 30.7 C Hum: 40.0 % Light: 24 %	PASS
FT-07	Return temperature to normal	Alarm stops	[SensorTask] Temp: 30.7 C Hum: 50.0 % Light: 63 % [SensorTask] Temp: 25.6 C Hum: 50.0 % Light: 63 % [AlarmTask] Alarm state changed -> NORMAL [SensorTask] Temp: 24.8 C Hum: 50.0 % Light: 63 % [SensorTask] Temp: 24.8 C Hum: 50.0 % Light: 63 %	PASS
FT-08	Trigger PIR	System is ACTIVE		PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE		PASS

Test ID	Input/Stimulus	Expected	Actual	Result
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE		PASS

Deliberate FreeRTOS Fault Experiments

Experiment 1 — Remove Blocking

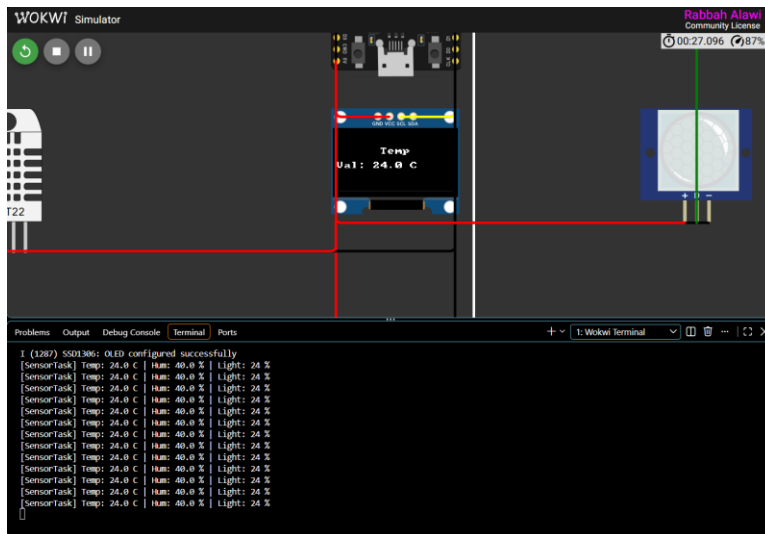
Removing InputTask's vTaskDelay() made it use the CPU continuously, preventing the idle task from running. This triggered ESP-IDF's Task Watchdog Timer, which identified InputTask as the cause.



This confirms that a task without a blocking call can run continuously and prevent even low-priority tasks from running. This triggers the watchdog, causing the terminal to stall and the OLED to freeze.

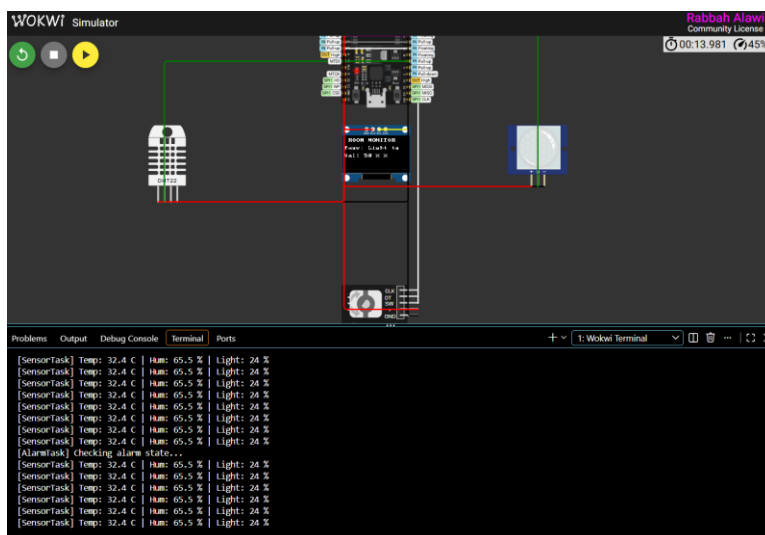
Experiment 2 — Change Priority

The encoder navigation gets delay and sometimes it skips pages compare before and the OLED flickering and updating gets faster.



Experiment 3 — Remove Mutex

The terminal printing gets too fast and in the lines “Checking alarm state...” randomly printing.



6. Static Code Analysis

Finding	File/Line	Cause	Resolution
pio check crashed before finishing	Whole project; first seen at system_state.cpp	ESP-IDF adds a huge number of compiler macros that overload cppcheck's limits	Ran cppcheck directly on just the project's own files, skipping ESP IDF's macros
No warnings found in the 8 source files	alarm.cpp, display.cpp, input.cpp, main.cpp, motion.cpp, rtos_objects.cpp, sensors.cpp, system_state.cpp	Clean result	This run couldn't see FreeRTOS/ESP IDF headers, so it only confirms no issues in the parts it could check.

Finding	File/Line	Cause	Resolution
unusedFunction warnings on getNextDisplayMode, getPreviousDisplayMode, evaluateSystemState	system_state.cpp:5, 15, 25	First two are false alarms, cppcheck checked files one at a time and missed calls from other files.	Suppressed unusedFunction in the direct cppcheck run since single file analysis makes this check unreliable in this build setup.

```

PS C:\Users\Lenovo\Documents\PlatformIO\Projects\bca152-freertos-multisensor> pio check -e esp32doit-devkit-v1
Checking esp32doit-devkit-v1 > cppcheck (platform: espressif32; board: esp32doit-devkit-v1; framework: espidf)
-----
Error: cppcheck failed to perform check! Please examine tool output in verbose mode.
===== [FAILED] Took 36.38 seconds =====

Environment      Tool      Status      Duration
-----
esp32doit-devkit-v1  cppcheck  FAILED      00:00:36.378

===== 1 failed, 0 succeeded in 00:00:36.378 =====
PS C:\Users\Lenovo\Documents\PlatformIO\Projects\bca152-freertos-multisensor>

```

```

PS C:\Users\Lenovo\Documents\PlatformIO\Projects\bca152-freertos-multisensor> "C:\Users\Lenovo\platformio\packages\tool-cppcheck\cppcheck.exe" --enable=warning,performance,portability --inline-suppr -i include src\alarm.cpp src\display.cpp src\input.cpp src\main.cpp src\motion.cpp src\rtos_objects.cpp src\sensors.cpp src\system_state.cpp
cpp src\system_state.cpp
cppcheck: Windows 64-bit binaries currently default to the 'win64' platform. Starting with Cppcheck 2.13 they will default to 'native' instead. Please specify '--platform=win64' explicitly if you rely on this.
Checking src\alarm.cpp ...
1/8 files checked 9% done
Checking src\display.cpp ...
2/8 files checked 30% done
Checking src\input.cpp ...
3/8 files checked 38% done
Checking src\main.cpp ...
4/8 files checked 46% done
Checking src\motion.cpp ...
5/8 files checked 58% done
Checking src\rtos_objects.cpp ...
6/8 files checked 63% done
Checking src\sensors.cpp ...
7/8 files checked 90% done
Checking src\system_state.cpp ...
8/8 files checked 100% done
PS C:\Users\Lenovo\Documents\PlatformIO\Projects\bca152-freertos-multisensor>

```

7. Engineering Discussion

Two debugging problem stand out

First, my Wokwi terminal produced zero output not even the boot banner despite a correctly built firmware; the cause turned out to be a missing \$serialMonitor wiring in diagram.json, found by comparing from Laboratory 0 initialization, which the on/off is displaying at terminal.

Second, my AlarmTask was originally an empty function that returned immediately, which FreeRTOS treats as fatal it silently crashed the whole system right after boot, which had been masking the serial monitor debugging the entire time.

Both taught me that "the code compiles" and "the code runs correctly" have very different meaning.

8. Conclusion

The system meets all functional requirements sensing, display, navigation, alarm, and state management verified through unit tests, functional tests, and fault experiments. The main thing to improve is making motion.cpp actually use the tested evaluateSystemState() function instead of duplicating its logic.

Required FreeRTOS Task Table

Task Name	Responsibility	Trigger/Period	Priority	IPC	Typical Blocked Condition
SensorTask	Reads DHT22 (temp/humidity) and LDR (light), evaluates alarm condition	2 s (vTaskDelayUntil)	2	Queue (displayQueue), Event Group (EVENT_ALARM)	Delay
DisplayTask	Renders current page on OLED	~100 ms loop	1	Queue (displayQueue, navQueue), Event Group (EVENT_ACTIVE)	Delay
InputTask	Polls rotary encoder for navigation	10 ms periodic	3	Queue (navQueue)	Delay
MotionTask	Polls PIR, manages ACTIVE/INACTIVE state	100 ms periodic	3	Event Group (g_systemEvents)	Delay
AlarmTask	Reads alarm state, drives buzzer	200 ms periodic	2	Event Group (EVENT_ALARM)	Delay

Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01 (Temperature measurement)	SensorTask	FT-01
FR-02 (Humidity measurement)	SensorTask	FT-02
FR-03 (Ambient-light measurement)	SensorTask	FT-03
FR-04 (Motion detection)	MotionTask	FT-08, FT-10
FR-05 (OLED display)	DisplayTask	FT-01–FT-03
FR-06 (Rotary encoder navigation)	InputTask	Unit navigation tests + FT-04/FT-05
FR-07 (Temperature alarm)	AlarmTask	Unit alarm tests + FT-06/FT-07
FR-08 (Activity state ACTIVE/INACTIVE)	MotionTask / system_state	Unit state tests
FR-09 (Automatic inactivity)	MotionTask	Unit state tests + FT-09
FR-10 (Automatic reactivation)	MotionTask	FT-10